

The Building Blocks of a Local RAG System

A from-scratch guide to APIs, async/await, scraping, embeddings, vector search, RAG, and running your own model with Ollama.

How everything fits together

Before the individual concepts, here is the whole picture, because these topics are not random — they are the parts of one machine.

Imagine you want to build a tool where you ask a question in plain English and get an answer based on specific documents you chose (a set of web pages, your company's docs, a pile of PDFs), not on whatever the AI happened to memorize during training. That tool is a RAG system, and here is the assembly line:

- **Scraping** grabs the raw text from web pages.
- That text gets split into chunks, and each chunk is turned into an **embedding** (a list of numbers representing its meaning).
- The embeddings go into storage.
- When a question comes in, you embed the question too, then run a **vector/similarity search** to find the chunks whose embeddings are closest to the question's embedding.
- You hand those chunks to a language model as context and ask it to answer — that handoff is the **RAG** step.
- The language model itself can run locally on your own machine via **Ollama**, so you don't need a paid API.
- **API calls** and **async/await** are the plumbing that connects all of this — how your code talks to the embedding service, the model, and the web.

Read top to bottom and each section builds on the last. Now the details.

1. API calls and async/await in JavaScript

What an API call is

An API (Application Programming Interface) is a defined way for one program to ask another program to do something or hand over some data. When your code makes an API call, it sends a request — usually over the internet using HTTP — to some server, and the server sends back a response.

A useful mental model: an API call is like ordering at a restaurant counter. You (your code) hand over an order slip with specifics (the request). The kitchen (the server) does work you can't see. Some time later,

your food comes back (the response). You don't need to know how the kitchen works — you just need to know how to place a valid order and what will come back.

A request typically has:

- **A URL** — the address you're sending to, e.g. `https://api.example.com/search`.
- **A method** — `GET` (fetch data), `POST` (send data / create something), and others like `PUT` and `DELETE`.
- **Headers** — metadata, like what format you're sending or an authentication key.
- **A body** — the actual data you're sending (usually only for `POST/PUT`), commonly formatted as JSON.

The response usually has a status code (`200` means OK, `404` means not found, `500` means the server broke) and a body (the data, again usually JSON).

The problem `async/await` solves

An API call takes time — maybe 50 milliseconds, maybe 3 seconds. That's an eternity for a computer. If your program simply froze and waited, doing nothing else until the response arrived, everything would grind to a halt. This is called blocking, and it's bad.

JavaScript handles slow operations asynchronously: it kicks off the request and keeps going, then deals with the result whenever it finally arrives. The trouble is that “deal with it whenever it arrives” used to make code a tangled mess of nested callbacks. `async/await` is modern JavaScript's clean solution.

How `async/await` actually works

Two keywords:

- **`async`** goes in front of a function. It marks that function as one that does asynchronous work and may need to wait.
- **`await`** goes in front of an operation that takes time. It means: pause this function here until the result comes back, but don't freeze the rest of the program.

Here's a real example fetching data from an API:

```
async function getUser(userId) {  
  // Send the request and WAIT for the response to come back.  
  const response = await fetch(`https://api.example.com/users/${userId}`);  
  
  // The response arrived, but we still need to WAIT while its  
  // JSON body is parsed into a usable object.  
  const data = await response.json();  
  
  return data;  
}  
  
// Calling it:  
async function main() {
```

```
const user = await getUser(42);
console.log(user.name);
}

main();
```

Walking through it:

- `fetch(...)` starts the API call. Because it's slow, we put `await` in front — the function politely pauses at that line.
- While paused, the JavaScript engine is free to run other code. Nothing is frozen.
- When the response arrives, execution resumes on the next line.
- `response.json()` is also asynchronous (reading and parsing the body takes time too), so it gets its own `await`.

Promises: what's underneath

`await` works on things called Promises. A Promise is an object that represents a value that isn't ready yet — a placeholder for a future result. It's in one of three states: pending (still working), fulfilled (succeeded, here's the value), or rejected (failed, here's the error). `await` unwraps a Promise: it waits for it to settle and gives you either the value or throws the error. You rarely need to touch Promises directly once you have `async/await`, but it helps to know that's the machinery `await` is operating.

Handling errors

Because `await` can throw when a request fails, you wrap risky calls in `try/catch`:

```
async function getUserSafely(userId) {
  try {
    const response = await fetch(`https://api.example.com/users/${userId}`);

    // fetch does NOT throw on 404/500 -- you must check yourself.
    if (!response.ok) {
      throw new Error(`Request failed with status ${response.status}`);
    }

    return await response.json();
  } catch (error) {
    console.error("Something went wrong:", error.message);
    return null;
  }
}
```

One gotcha worth memorizing: `fetch` only rejects on network failures (no connection, DNS failure). A `404` or `500` still counts as a “successful” fetch as far as the Promise is concerned — the server did respond, just with bad news. That's why you check `response.ok` manually.

Doing several calls at once

If you have three independent API calls, awaiting them one after another wastes time — you wait for the first to finish before even starting the second. `Promise.all` runs them concurrently:

```
async function getThreeUsers() {  
  const [a, b, c] = await Promise.all([  
    getUser(1),  
    getUser(2),  
    getUser(3),  
  ]);  
  return [a, b, c];  
}
```

All three requests fire at roughly the same moment, and `await` waits for the slowest one. If you have 100 pages to scrape or 100 chunks to embed, this pattern (in controlled batches) is how you keep things fast.

2. Scraping: fetching a page and pulling out the text

Scraping means writing code that fetches a web page's raw HTML and then extracts the specific pieces of text you actually want, discarding the navigation bars, ads, footers, and markup.

When you visit a page in a browser, the browser downloads HTML — a big string of tags like `<h1>`, `<p>`, `<div>` — and renders it into the nice visual page you see. A scraper skips the rendering. It grabs that same HTML string and treats it as data to be mined.

The two steps

Step one: fetch the HTML. This is just an API call to the page's URL, and the “response” is the HTML.

Step two: parse and extract. Raw HTML looks like this:

```
<html>  
  <body>  
    <nav>Home | About | Contact</nav>  
    <article>  
      <h1>Why Cats Sleep So Much</h1>  
      <p>Cats sleep 12 to 16 hours a day because...</p>  
    </article>  
    <footer>Copyright 2026</footer>  
  </body>  
</html>
```

You almost never want the whole thing. You want the article title and the paragraph text — not “Home | About | Contact” or the copyright line. Parsing turns that string into a navigable tree of elements so you can say “give me the text inside the `<article>` tag” or “give me every `<p>`.”

A concrete example in Node.js

Using fetch (built into modern Node) plus Cheerio, a popular library that lets you query HTML the way jQuery queries a page:

```
import * as cheerio from "cheerio";

async function scrapeArticle(url) {
  // Step 1: fetch the raw HTML.
  const response = await fetch(url);
  const html = await response.text(); // .text() gives the raw string

  // Step 2: load it into Cheerio and query it.
  const $ = cheerio.load(html);

  const title = $("h1").first().text().trim();

  // Grab every paragraph inside <article>, get its text, join them.
  const paragraphs = $("article p")
    .map((i, el) => $(el).text().trim())
    .get()
    .join("\n\n");

  return { title, body: paragraphs };
}
```

The `$("article p")` part is a CSS selector — the same kind of pattern used to style pages. `"article p"` means “every `<p>` that lives inside an `<article>`.” Selectors are how you point at exactly the text you want.

Things that make scraping harder in practice

- **JavaScript-rendered pages.** Many modern sites send a nearly-empty HTML shell and then build the content with JavaScript in the browser. A plain fetch gets the empty shell. For these you need a headless browser like Playwright or Puppeteer, which actually runs the page's JavaScript before you read the result.
- **Messy, inconsistent structure.** Every site is built differently, so selectors that work on one won't work on another. Real scrapers often need per-site tweaking.
- **Rate limits and politeness.** Hammering a server with hundreds of rapid requests is rude and can get you blocked. Space out requests, and check the site's robots.txt and terms of service — some sites disallow scraping.
- **Cleaning.** Even after extraction you'll often have stray whitespace, leftover markup, or boilerplate to strip before the text is usable.

For a RAG system, scraping is how you gather the source material. The cleaned text you get out of this step is what feeds into the next one.

3. Embeddings: turning text into numbers that capture meaning

An embedding is a way of representing a piece of text as a list of numbers — a vector — such that texts with similar meaning end up with similar numbers.

That last clause is the whole point. Anyone can turn text into numbers (ASCII codes do that). What makes an embedding special is that the numbers encode meaning, so that “the cost was very high” and “it was quite expensive” — which share almost no words — land close together, while “the cost was very high” and “the cat was very fluffy” land far apart despite sharing more words.

What an embedding vector looks like

A single embedding is a fixed-length list of decimal numbers, often a few hundred to a couple thousand of them:

```
"a happy dog" -> [0.021, -0.834, 0.115, 0.402, ..., -0.077]
                  (e.g. 768 or 1536 numbers, always the same length
                   for a given model)
```

Each number is a coordinate along one dimension. If a vector has 768 numbers, it's a point in a 768-dimensional space. You can't picture 768 dimensions, but the intuition from 2D holds: points that are close together represent similar meanings.

The geometric intuition

Picture a simple 2D map where meaning is encoded by position. “Dog,” “puppy,” and “wolf” cluster in one region; “car,” “truck,” and “bus” cluster in another far away. “Cat” sits near the dog cluster but not on top of it. That clustering-by-meaning is exactly what a real embedding does, just in far more dimensions so it can capture far more nuance.

A famous property that falls out of good embeddings: relationships become directions. The vector arithmetic king – man + woman lands near queen, because the model has learned that the “royalty” and “gender” aspects of meaning live along consistent directions in the space.

How you actually get one

You don't compute embeddings by hand — you send text to a model trained for it. That's an API call (local or remote). With Ollama running locally:

```
async function embed(text) {
  const response = await fetch("http://localhost:11434/api/embeddings", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      model: "nomic-embed-text", // an embedding model you've pulled
      prompt: text,
    }),
  });
  const data = await response.json();
  return data.embedding; // the array of numbers
}
```

```
}
```

Note this is exactly the API-call machinery from Section 1: a **POST** request, a JSON body, await on the response. Every piece connects.

The key rule

Always embed everything you'll compare using the same model. Vectors from different models live in different, incompatible spaces — comparing them is meaningless, like comparing a temperature in Celsius to a distance in miles. In a RAG system you embed your document chunks and your incoming questions with the same embedding model so the numbers are comparable.

4. Vector / similarity search: finding the closest chunks

Once your text chunks are stored as embeddings, vector search (also called similarity search or semantic search) answers this question: given a new embedding — say, an embedded question — which of my stored embeddings are the most similar to it?

Because similar meaning means nearby vectors, “find the most relevant chunks for this question” reduces to “find the stored vectors closest to the question's vector.” You've turned a fuzzy language problem into a concrete geometry problem: nearest-neighbor search.

Measuring similarity: cosine similarity

The most common measure is cosine similarity, which looks at the angle between two vectors rather than the raw distance. Two vectors pointing in the same direction have cosine similarity **1.0** (maximally similar); perpendicular vectors give **0**; opposite directions give **-1**. Direction is used because it captures meaning more robustly than raw magnitude.

In code, cosine similarity is just a dot product divided by the two lengths:

```
function cosineSimilarity(a, b) {  
  let dot = 0, magA = 0, magB = 0;  
  for (let i = 0; i < a.length; i++) {  
    dot += a[i] * b[i];  
    magA += a[i] * a[i];  
    magB += b[i] * b[i];  
  }  
  return dot / (Math.sqrt(magA) * Math.sqrt(magB));  
}
```

The simplest possible search

If you only have a few hundred chunks, you can search by brute force: compare the question's embedding against every stored embedding, sort by similarity, keep the top few.

```
function search(queryEmbedding, storedChunks, topK = 3) {  
  return storedChunks  
    .map(chunk => ({  
      text: chunk.text,  
      score: cosineSimilarity(queryEmbedding, chunk.embedding),  
    }))  
    .sort((a, b) => b.score - a.score) // highest similarity first  
    .slice(0, topK); // keep the best few  
}
```

topK is how many chunks you retrieve — typically 3 to 8. Those are the chunks most relevant to the question.

Why keyword search isn't enough

The reason to bother with all this instead of a plain text search: keyword search only finds exact word matches. If your document says “feline sleep patterns” and someone asks about “how much do cats nap,” keyword search finds nothing — no shared words. Vector search finds it anyway, because the meanings are close. This is the core advantage of semantic search.

Scaling up: vector databases

Brute force works for hundreds or a few thousand chunks. For millions, comparing against every vector is too slow. Vector databases — such as Chroma, FAISS, Pinecone, Qdrant, or Weaviate — store embeddings and use clever indexing (approximate nearest neighbor algorithms) to find the closest vectors in milliseconds without checking every single one. When you're starting out, an in-memory array and the brute-force loop above are perfectly fine; reach for a vector database when volume demands it.

5. RAG: retrieval-augmented generation

RAG stands for Retrieval-Augmented Generation. It's a technique for making a language model answer using specific text you provide, rather than relying solely on what it memorized during training.

The problem RAG solves

A language model on its own has real limitations. It only knows what was in its training data, so it's unaware of anything newer, private, or niche. It can't cite where an answer came from. And when it doesn't know something, it may hallucinate — produce a fluent, confident, wrong answer.

Asking a model a question “cold” — just the question, nothing else — leans entirely on its memory and invites all these problems.

The RAG idea

Instead of asking cold, you do two steps:

- **Retrieve.** Find text relevant to the question from your own collection of documents (this is exactly the embedding + vector search from Sections 3 and 4).
- **Generate.** Put that retrieved text into the prompt as context, then ask the model to answer using it.

The model is no longer guessing from memory. It's reading the relevant passages you handed it and answering from those — like the difference between asking someone a trivia question from memory versus letting them look it up in the right book first.

What the prompt looks like

The magic is unglamorous. You build a prompt that stitches the retrieved chunks together with the question and an instruction:

```
function buildRagPrompt(question, retrievedChunks) {
  const context = retrievedChunks.map(c => c.text).join("\n\n---\n\n");

  return `Answer the question using ONLY the context below.
If the answer isn't in the context, say you don't know.

Context:
${context}

Question: ${question}

Answer: `;
}
```

The instruction to use only the context and to admit ignorance is what curbs hallucination and keeps the model grounded in your sources.

The full RAG pipeline, end to end

Putting all the pieces together:

```
async function answerWithRag(question, storedChunks) {
  // 1. Embed the incoming question (same model used for the chunks).
  const questionEmbedding = await embed(question);

  // 2. Vector search: find the most relevant stored chunks.
  const relevant = search(questionEmbedding, storedChunks, 4);

  // 3. Build a prompt that includes those chunks as context.
  const prompt = buildRagPrompt(question, relevant);

  // 4. Ask the language model to answer from that context.
  const answer = await askLLM(prompt); // e.g. a call to Ollama

  return answer;
}
```

Two distinct phases run behind this. There's an offline indexing phase, done once up front: scrape documents, split them into chunks, embed each chunk, and store the embeddings. Then there's the query phase, run every time a question comes in: embed the question, search, build the prompt, generate the answer. The function above is the query phase; the indexing phase is what fills `storedChunks` in the first place.

A note on chunking

Documents get split into chunks — passages of maybe a few hundred words — rather than embedded whole, for two reasons. A single embedding can only capture so much meaning, so one vector for an entire book would be hopelessly vague. And you want to hand the model just the relevant passage, not an entire document, both to stay within its context limit and to keep it focused. Good chunking (sensible sizes, a little overlap between adjacent chunks so ideas aren't cut mid-thought) has an outsized effect on RAG quality. It's often where the most tuning happens.

6. Ollama: running a model on your own machine

Ollama is a tool that lets you download and run large language models locally, on your own computer, instead of calling a paid cloud API. It handles the messy parts — downloading model weights, setting them up, and exposing a simple local API — so running a model becomes a couple of commands. This is what lets a RAG system run entirely on your own hardware, with your data never leaving your machine.

Installing Ollama

- **macOS and Windows:** download the installer from ollama.com/download and run it. It installs an app that runs Ollama in the background.
- **Linux:** run the official install script:

```
curl -fsSL https://ollama.com/install.sh | sh
```

Once installed, Ollama runs a local server, by default at <http://localhost:11434>. That's the address the earlier embedding and LLM code was talking to.

Verify it's working:

```
ollama --version
```

Pulling a model

Models aren't included by default — you download (“pull”) the ones you want. To get Meta's Llama 3:

```
ollama pull llama3
```

This downloads the model weights, which can be several gigabytes, so it takes a bit the first time. Ollama has many models available — different families (Llama, Mistral, Gemma, Qwen, and others) and different sizes. Smaller models run faster and need less memory but are less capable; larger ones are the reverse. For a RAG system you'll typically pull two kinds: a chat model like `llama3` to generate answers, and an embedding model like `nomic-embed-text` to create the vectors.

```
ollama pull nomic-embed-text
```

You can list what you've downloaded:

```
ollama list
```

Sending a test prompt from the terminal

The quickest way to check a model works is the interactive run command:

```
ollama run llama3
```

This drops you into a chat prompt where you type a message and get a reply:

```
>>> Explain what an embedding is in one sentence.
An embedding is a numerical representation of data -- such as text --
that captures its meaning as a list of numbers, so that similar
items have similar representations.

>>> /bye
```

Type `/bye` (or press Ctrl-D) to exit. You can also pass a one-off prompt directly without entering interactive mode:

```
ollama run llama3 "Write a haiku about databases."
```

Talking to Ollama from code

For an actual application, you use Ollama's local HTTP API — the same `localhost:11434` server — which is just more of the API-call machinery from Section 1:

```
async function askLLM(prompt) {
  const response = await fetch("http://localhost:11434/api/generate", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      model: "llama3",
      prompt: prompt,
      stream: false,          // get the whole answer at once
    }),
  });
  const data = await response.json();
  return data.response;      // the model's generated text
}
```

```
| }
```

That askLLM function is exactly the one called at the end of the RAG pipeline in Section 5. With Ollama serving both `llama3` for generation and `nomic-embed-text` for embeddings, every piece of the system — scraping, embedding, searching, generating — can run on your own machine.

Putting it all together

You now have every building block:

- **API calls** and **async/await** — the plumbing for talking to any service over the network without freezing your program.
- **Scraping** — fetching web pages and extracting the text you care about, to build your source material.
- **Embeddings** — turning that text into meaning-capturing vectors.
- **Vector search** — finding the stored chunks most similar to a question.
- **RAG** — retrieving relevant chunks and handing them to a model as context so it answers from your sources instead of guessing.
- **Ollama** — running the models locally so the whole thing is yours end to end.

A minimal project that ties them together looks like this: scrape a handful of pages, chunk and embed the text with `nomic-embed-text` via Ollama, store the vectors in a plain array, then for each question embed it, brute-force cosine-search for the top few chunks, build a context prompt, and send it to `llama3`. That's a complete, working RAG system running entirely on your own machine — and every part of it is one of the sections above.

A sensible learning order: get Ollama installed and chat with `llama3` in the terminal first (Section 6) so you have a working model to talk to; then wire up a single API call from code (Section 1); then build outward from there. Each concept is small on its own — the power is in how they connect.